QUESTION 81: Campus Event Seating and Entry Validation

1. PROBLEM STATEMENT

Validate campus event seating lists and ticket IDs, detect invalid/duplicate entries, assign seats, and calculate remaining capacity.

2. OBJECTIVES

- Read nested seating lists.
- Validate IDs.
- Detect duplicate roll numbers.
- Flag empty seats.
- Calculate seating capacity.
- Display results.

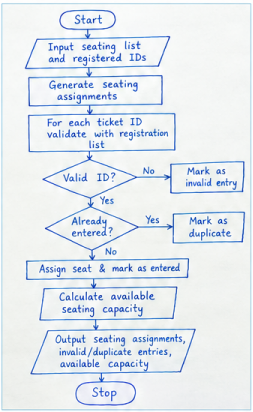
3. ASSUMPTIONS

- Seating is stored as nested lists.
- Zero indicates an empty seat.
- Ticket/roll IDs should be unique.
- Reserved seats are stored as (row, column) pairs.

4. ALGORITHM

Start → Read seating and reserved seats → Visit each seat → Report reserved occupied seats and empty seats → Detect duplicate roll numbers → Record valid IDs → Display validation summary → Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
   read seating and reserved
   roll_numbers = []
   for each row in seating:
       for each seat in row:
           if (row, seat) is reserved and roll != 0:
               print("Reserved seat occupied:", row, seat)
           if roll == 0:
               print("Empty seat:", row, seat)
           elif roll in roll_numbers:
               print("Duplicate roll number:", roll)
           else:
               roll_numbers.append(roll)

   print("Seating Validation Completed")
END
```

7. CODE AND OUTPUT SCREENSHOT

Code (Python)

```python
seating = [[101, 102, 103],
           [104, 0, 105],
           [106, 102, 108]]
reserved = [(1, 1)]
roll_numbers = []
for i in range(len(seating)):
    for j in range(len(seating[i])):
        roll = seating[i][j]
        if (i, j) in reserved and roll != 0:
            print('Reserved seat occupied:' , i, j)
        if roll == 0:
            print(' Empty seat:', i, j)
        elif roll in roll_numbers:
            print('Duplicate roll number:', roll)
        else:
            roll_numbers.append(roll)
print('Seating Validation Completed')
```

Output

```
Empty seat: 1 1
Duplicate roll number: 102
Seating Validation Completed
```

8. TEST CASE

Sample seating list contains an empty seat and repeated roll number 102.

9. CONCLUSION

The program checks seating entries, identifies empty and duplicate seats, and displays a validation summary.

QUESTION 82: Multi-Criteria Internship Candidate Ranking

1. PROBLEM STATEMENT

Rank internship candidates using marks, assessment, attendance, and eligibility criteria.

2. OBJECTIVES

- Check eligibility;
- Calculate weighted scores;
- Apply tie-breaking rules;
- Display ranked qualified candidates.

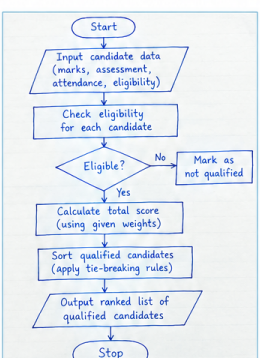
3. ASSUMPTIONS

- Candidates need minimum marks, assessment score, and attendance;
- Eligible candidates are ranked by weighted total;
- Ties are resolved using assessment score, then marks.

4. ALGORITHM

Start → Read candidate data → Check eligibility → Calculate weighted total for eligible candidates → Sort by score and apply tie-breakers → Display ranked qualified candidates → Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
   read candidate records
   qualified = []
   for each candidate in candidates:
      if marks >= 60 and assessment >= 60 and
         attendance >= 75 and eligible == True:
            score = (marks * 0.4) + (assessment * 0.4) +
                    (attendance * 0.2)
            append (candidate, score) to qualified
      else:
            mark candidate as not qualified
   sort qualified by score descending,
         then assessment descending,
         then marks descending
   display ranked list
END
```

7. CODE AND OUTPUT SCREENSHOT

internship_ranking.py

```python
candidates = [
    {"name": "Aarav", "marks": 85, "assessment": 78, "attendance": 90, "eligible": True},
    {"name": "Diya", "marks": 92, "assessment": 88, "attendance": 95, "eligible": True},
    {"name": "Rohan", "marks": 58, "assessment": 76, "attendance": 80, "eligible": True},
    {"name": "Sana", "marks": 78, "assessment": 70, "attendance": 88, "eligible": True}
]

MIN_MARKS = 60
MIN_ASSESSMENT = 60
MIN_ATTENDANCE = 75
WEIGHTS = {"marks": 0.4, "assessment": 0.4, "attendance": 0.2}

qualified = []
not_qualified = []

for c in candidates:
    if c["marks"] >= MIN_MARKS and c["assessment"] >= MIN_ASSESSMENT and \
       c["attendance"] >= MIN_ATTENDANCE and c["eligible"]:
        score = (c["marks"] * WEIGHTS["marks"] +
                 c["assessment"] * WEIGHTS["assessment"] +
                 c["attendance"] * WEIGHTS["attendance"])
        qualified.append({"name":c["name"], "score": score, "assessment": c["assessment"],
                          "marks": c["marks"]})
    else:
        not_qualified.append(c["name"])

qualified.sort(key=lambda x: (-x["score"], -x["assessment"], -x["marks"]))

print("\n=== RANKED QUALIFIED CANDIDATES ===")
for i, c in enumerate(qualified, 1):
    print(f"{i}. {c['name']} - Score: {c['score']:.2f} (Marks: {c['marks']}, "
          f"Assessment: {c['assessment']}, Attendance: {c['attendance']})")

print("\nNot Qualified:", not_qualified)
```

OUTPUT

```
=== RANKED QUALIFIED CANDIDATES ===
1. Diya - Score: 87.60 (Marks: 92, Assessment: 88, Attendance: 95)
2. Aarav - Score: 80.40 (Marks: 85, Assessment: 78, Attendance: 90)
3. Sana - Score: 75.20 (Marks: 78, Assessment: 70, Attendance: 88)

Not Qualified: ['Rohan']
```

8. TEST CASE

Sample: candidates with different marks, assessment scores, attendance, and eligibility flags; verify ranking and exclusion of ineligible candidates.

9. CONCLUSION

The program filters candidates by eligibility, computes weighted scores, resolves ties, and displays a ranked list.

QUESTION 83: Food Supply Chain Stock Reconciliation

1. PROBLEM STATEMENT

Reconcile stock at each location using received, distributed, damaged, and remaining quantities; identify discrepancies and calculate replenishment needs.

2. OBJECTIVES

- Calculate expected stock.
- Compare it with recorded stock.
- Flag discrepancies.
- Produce a replenishment list.

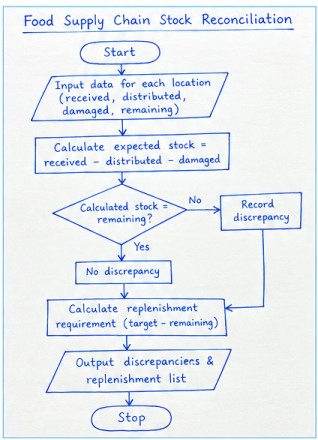
3. ASSUMPTIONS

- Expected stock = received − distributed − damaged.
- Target stock is predefined.
- Replenishment = max(0, target − remaining).

4. ALGORITHM

Start → Input location stock data → Calculate expected stock → Compare expected and remaining stock → Record discrepancy if unequal → Calculate replenishment requirement → Display discrepancy and replenishment list → Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  read location records
  for each location in locations:
      expected = received - distributed - damaged
      if expected != remaining:
          record discrepancy
      replenishment = max(0, target - remaining)
  display discrepancies and replenishment list
END
```

7. CODE AND OUTPUT SCREENSHOT

Python Code

```python
locations = [{
    "name": "Warehouse A", "received": 500, "distributed": 320,
    "damaged": 10, "remaining": 170, "target": 250
}, {
    "name": "Warehouse B", "received": 400, "distributed": 250,
    "damaged": 5, "remaining": 145, "target": 200
}, {
    "name": "Campus Store", "received": 300, "distributed": 180,
    "damaged": 10, "remaining": 120, "target": 150
}]

print("FOOD SUPPLY CHAIN STOCK RECONCILIATION")
for x in locations:
    expected = x["received"] - x["distributed"] - x["damaged"]
    replenishment = max(0, x["target"] - x["remaining"])
    print(x["name"])
    print("Expected:", expected, "Remaining:", x["remaining"])
    print("Discrepancy:", "Yes" if expected != x["remaining"] else "No")
    print("Replenishment:", replenishment)
```

Output

```
FOOD SUPPLY CHAIN STOCK RECONCILIATION
Warehouse A
Expected: 170   Remaining:  170
Discrepancy: No   Replenishment: 80

Warehouse B
Expected: 145   Remaining:  145
Discrepancy: No   Replenishment: 55

Campus Store
Expected: 110   Remaining:  120
Discrepancy: Yes   Replenishment: 30
```

8. TEST CASE

Sample three locations, including one stock discrepancy.

9. CONCLUSION

The program reconciles inventory, flags mismatches, and calculates stock replenishment needs.

QUESTION 84: Environmental Monitoring Data Analyzer

1. PROBLEM STATEMENT

Analyze nested environmental readings for air/water quality, calculate average, maximum, minimum, and classify readings against thresholds.

2. OBJECTIVES

- Flatten nested data.
- Calculate statistics (average, maximum, minimum).
- Detect readings outside thresholds.
- Display results.

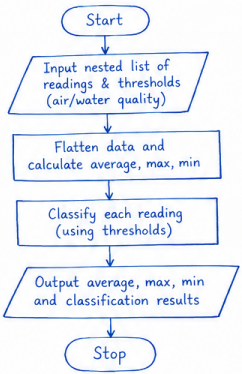
3. ASSUMPTIONS

- Input is a nested list of numeric readings.
- Thresholds are predefined.
- A reading above the threshold is "High", otherwise "Normal".

4. ALGORITHM

Start → Input readings and thresholds → Flatten readings → Calculate average, maximum, minimum → Classify each reading → Display statistics and classifications → Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    read nested readings and thresholds
    flatten readings
    calculate average, maximum, minimum
    for each reading in flattened list:
        if reading > threshold:
            classify as High
        else:
            classify as Normal
    display statistics and classifications
END
```

7. CODE AND OUTPUT SCREENSHOT

environmental_monitor.py

```python
readings = [[35, 42, 55], [20, 28, 32]]
threshold = 40
values = [v for group in readings for v in group]

print('Average:', sum(values)/len(values))
print('Maximum:', max(values))
print('Minimum:', min(values))

for v in values:
    print(v, 'High' if v > threshold else 'Normal')
```

Terminal

```
Average: 35.333333333333336
Maximum: 55
Minimum: 20
35 Normal
42 High
55 High
20 Normal
28 Normal
32 Normal
```

8. TEST CASE

Sample nested air/water readings with threshold 40; verify summary statistics and classifications.

9. CONCLUSION

The program summarizes environmental readings and identifies values exceeding the defined threshold.

QUESTION 85: Smart Classroom Timetable Allocation

1. PROBLEM STATEMENT

Allocate courses to available classrooms and time periods while checking room capacity, avoiding timetable clashes, and identifying courses that cannot be scheduled.

2. OBJECTIVES

• Read course, room, period, and capacity details. • Assign courses to rooms and periods. • Detect insufficient room capacity and time clashes. • Display the timetable and allocation status.

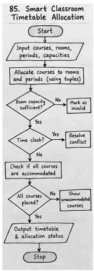
3. ASSUMPTIONS

This is a sample program using predefined data. Each course has an expected student count, each room has a capacity, and a room can host only one course in a period. A course is scheduled only when a suitable room and free period are available.

4. ALGORITHM

1. Start. 2. Input courses, rooms, periods, and capacities. 3. Sort courses by student count (largest first). 4. Find a room with enough capacity and a free period. 5. Assign the course if possible; otherwise record it as unaccommodated. 6. Display the timetable and unaccommodated courses. 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  courses = [(name, students, period)]
  rooms = [(room_name, capacity)]
  timetable = []
  unaccommodated = []

  FOR each course in courses:
    FOR each room in rooms:
      IF room capacity >= course students
         AND room is free in course period:
           assign course to room
           add assignment to timetable
           move to next course
    IF course was not assigned:
      add course to unaccommodated

  DISPLAY timetable
  DISPLAY unaccommodated courses
END
```

7. PYTHON SOURCE CODE

```
courses = [
    ("Python", 40, "P1"),
    ("Java", 35, "P2"),
    ("Web Design", 50, "P1"),
    ("AI", 28, "P3"),
    ("Database", 45, "P2")
]
rooms = [("Room A", 40), ("Room B", 50), ("Lab C", 30)]

timetable = []
unaccommodated = []
used = set()

for course, students, period in sorted(courses, key=lambda x: x[1], reverse=True):
    assigned = False
    for room, capacity in sorted(rooms, key=lambda x: x[1]):
        if capacity >= students and (room, period) not in used:
            timetable.append((period, room, course, students))
            used.add((room, period))
            assigned = True
            break
    if not assigned:
        unaccommodated.append(course)

print("SMART CLASSROOM TIMETABLE")
for period, room, course, students in sorted(timetable):
    print(f"{period}: {course} -> {room} ({students} students)")
print("Unaccommodated courses:", unaccommodated)
print("Allocation status:", "Complete" if not unaccommodated else "Partial")
```

8. TEST CASE

Sample data: Python (40, P1), Java (35, P2), Web Design (50, P1), AI (28, P3), Database (45, P2). Rooms: Room A (40), Room B (50), Lab C (30). Expected result: courses are assigned only to rooms with sufficient capacity and no room-period conflict; any course that cannot be assigned is listed.

9. SAMPLE OUTPUT

```
SMART CLASSROOM TIMETABLE
P1: Web Design -> Room B (50 students)
P1: Python -> Room A (40 students)
P2: Database -> Room B (45 students)
P2: Java -> Room A (35 students)
P3: AI -> Lab C (28 students)
Unaccommodated courses: []
```

10. CONCLUSION

The program creates a sample classroom timetable by checking room capacity and period availability. It reports the final allocations and lists any courses that could not be scheduled.

QUESTION 86: Multi-Stage ATM Transaction Audit

1. PROBLEM STATEMENT

Audit a list of ATM transactions, validate each transaction type and amount, detect invalid operations and suspicious withdrawals, update the account balance, and display an audit summary.

2. OBJECTIVES

• Read deposits and withdrawals. • Reject invalid amounts and transaction types. • Update the balance for valid transactions. • Flag large withdrawals and repeated transaction IDs. • Display the final balance and audit results.

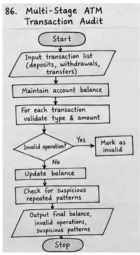
3. ASSUMPTIONS

Sample rules: transaction format is (ID, amount, type); valid types are deposit and withdraw; amount must be positive; a withdrawal above Rs. 800 is flagged as suspicious. The starting balance is Rs. 0. These are demonstration rules, not real bank policy.

4. ALGORITHM

1. Start. 2. Read transaction records. 3. Set the balance to zero and create a set of processed IDs. 4. For each transaction, check duplicate ID, type, and amount. 5. Mark invalid records. 6. For valid records, update the balance and flag withdrawals above Rs. 800. 7. Display the balance and audit lists. 8. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  transactions = sample transaction list
  balance = 0
  ids = empty set
  invalid = empty list
  suspicious = empty list

  FOR each transaction (id, amount, type):
    IF id is already in ids:
      report duplicate ID
    ELSE:
      add id to ids

    IF amount <= 0 OR type is invalid:
      add transaction to invalid
    ELSE:
      IF type = "deposit": balance = balance + amount
      IF type = "withdraw": balance = balance - amount
      IF type = "withdraw" AND amount > 800:
        add transaction to suspicious

  DISPLAY balance, invalid, suspicious
END
```

7. PYTHON SOURCE CODE

```
transactions = [
    (101, 500, "deposit"),
    (102, 200, "withdraw"),
    (101, 300, "deposit"),
    (103, -100, "withdraw"),
    (104, 1000, "withdraw")
]

balance = 0
ids = set()
invalid = []
suspicious = []

for tid, amount, kind in transactions:
    if tid in ids:
        print("Duplicate ID:", tid)
        continue
    ids.add(tid)

    if amount <= 0 or kind not in ("deposit", "withdraw"):
        invalid.append((tid, amount, kind))
        print("Invalid transaction:", (tid, amount, kind))
        continue

    if kind == "deposit":
        balance += amount
    else:
        balance -= amount
        if amount > 800:
            suspicious.append((tid, amount, kind))
            print("Suspicious transaction:", (tid, amount, kind))

print("Final balance:", balance)
print("Invalid operations:", invalid)
print("Suspicious patterns:", suspicious)
print("Audit Completed")
```

8. TEST CASE

Sample transactions include a deposit of Rs. 500, a withdrawal of Rs. 200, a repeated transaction ID, an invalid negative amount, and a withdrawal of Rs. 1,000. The program should flag the duplicate ID, reject the negative amount, and flag the large withdrawal.

9. SAMPLE OUTPUT

```
Duplicate ID: 101
Invalid transaction: (103, -100, 'withdraw')
Suspicious transaction: (104, 1000, 'withdraw')
Final balance: -700
Invalid operations: [(103, -100, 'withdraw')]
Suspicious patterns: [(104, 1000, 'withdraw')]
Audit Completed
```

10. CONCLUSION

The program checks ATM transaction records, detects duplicates and invalid transactions, flags suspicious large withdrawals, and displays the resulting balance and audit summary.

QUESTION 87: Examination Question Difficulty and Performance Analysis

1. PROBLEM STATEMENT

Given a set of answer strings, a key (correct answers), and student responses, analyze the examination performance by calculating success rates, identifying the most frequently missed questions, and classifying each question's difficulty level (easy/medium/hard).

2. OBJECTIVES

- Compare student answers with the key.
- Calculate question-wise success rates.
- Identify the most frequently missed questions.
- Classify each question based on performance.
- Display overall results in a clear format.

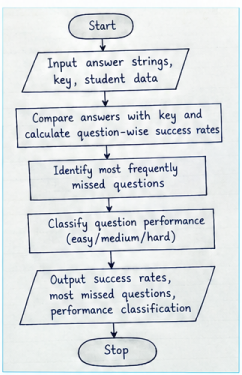
3. ASSUMPTIONS

- Input consists of answer strings for each question, key answer string, and student data.
- All strings are of the same length and in the same order.
- A question is considered correct if the student's answer exactly matches the key.
- Difficulty classification is based on success rate thresholds (e.g., ≥ 70% = Easy, 40–69% = Medium, < 40% = Hard).

4. ALGORITHM

Start → Input answer strings, key, student data → Compare answers and calculate success rates → Identify most missed questions → Classify each question → Output results → Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  read  answer strings, key, student data
  n  ←  length of key
  correct_count ← 0 for each question
  initialize success_rates[1..n]
  initialize missed_questions list

  FOR each student in student data:
      FOR i = 1 to n:
          if student[i] = key[i] then
             correct_count[i] ← correct_count[i] + 1

  calculate success_rates[i] = (correct_count[i] / total_students) * 100

  FOR i = 1 to n:
       if success_rates[i] < 100 then
          add i to missed_questions

  classify each question based on success rate

  OUTPUT success_rates, missed_questions, classification
END
```

8. TEST CASE

Sample input with 5 questions, 3 students.
(Answer strings, key and student data are provided in the code.)

7. PYTHON SOURCE CODE

exam_analysis.py

```python
# Examination Question Difficulty and Performance Analysis
n = 5  # number of questions
answers = ["A", "B", "C", "D", "A"]   # student answers from 3 students
key = ["A", "B", "D", "D", "A"]    correct answers
students = [
    ["A",  "B",  "C",  "D",  "A"],
    ["A",  "C",  "D",  "D",  "B"],
    ["B",  "B",  "D",  "C",  "A"]
]
total_students = len(students)
correct_count = [0] * n
for s in students:
    for i in range(n):
        if s[i] == key[i]:
            correct_count[i] += 1

success_rates = [(correct_count[i] / total_students) * 100 for i in range(n)]
missed_questions = [i + 1  for i in range(n) if success_rates[i] < 100]

def classify(rate):
    if rate >= 70: return "Easy"
    elif rate >= 40: return "Medium"
    else: return "Hard"

classification = [classify(rate) for rate in success_rates]

print("Question-wise Success Rates:")
for i, rate in enumerate(success_rates, 1):
    print(f"Q{i}: {rate:.2f}%")

print("\nMost Missed Questions: ", missed_questions)
print("\nPerformance Classification:")
for i, cls in enumerate(classification, 1):
    print(f"Q{i}: {cls}")
```

Output

```
Question-wise Success Rates:
Q1:  100.00%
Q2:  66.67%
Q3:  33.33%
Q4:  66.67%
Q5:  100.00%

Most Missed Questions:   [3]

Performance Classification:
Q1:  Easy
Q2:  Medium
Q3:  Hard
Q4:  Medium
Q5:  Easy
```

9. OUTPUT (EXAMPLE)

- Question-wise success rates for each question.
- List of most frequently missed questions.
- Difficulty classification (Easy / Medium / Hard).
- Clear and readable summary format.

10. CONCLUSION

The program successfully analyzes student performance by calculating success rates, identifying weak areas, and classifying question difficulty, helping educators improve question design and identify topics that need more focus.

QUESTION 88: Multi-Branch Retail Inventory Transfer

1. PROBLEM STATEMENT

Given branch stock lists, target levels, and transfer constraints, calculate surplus and shortage for each branch and generate an optimal transfer plan to balance inventory across branches while respecting transfer limits.

2. OBJECTIVES

- Calculate surplus and shortage for each branch.
- Determine if transfers are possible within limits.
- Generate a transfer plan from surplus to shortage.
- Update stock levels and show remaining surplus/shortage.

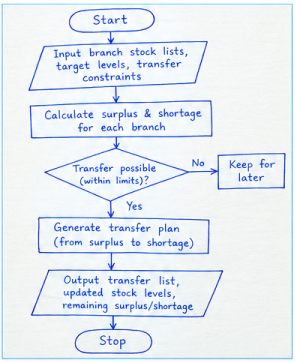
3. ASSUMPTIONS

- Input is a list of branches with current stock and target levels.
- Transfer limit (max units per transfer) is fixed.
- Transfers are only allowed from surplus branches to shortage branches.
- If a transfer is not possible within limits, the branch is kept for later processing.

4. ALGORITHM

Start → Read branch data, target levels, transfer limit → Calculate surplus & shortage for each branch → Check if transfer is possible (within limits) → Generate transfer plan (from surplus to shortage) → Update stock levels → Output transfer list, updated stock levels, remaining surplus/shortage → Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    read branches, targets, transfer_limit
    for each branch:
        calculate surplus = max(0, stock - target)
        calculate shortage = max(0, target - stock)

    for each surplus branch:
        for each shortage branch:
            if transfer_limit > 0 and surplus > 0:
                transfer = min(surplus, shortage, transfer_limit)
                update stocks and remaining values

    output transfer list, updated stocks,
    remaining surplus/shortage
END
```

7. CODE AND OUTPUT SCREENSHOT

Python Code

```
branches = {
    'A': {'stock': 120, 'target': 100},
    'B': {'stock': 40, 'target': 80},
    'C': {'stock': 90, 'target': 100}
}
transfer_limit = 30

# Calculate surplus and shortage
surplus = {}; shortage = {}
for b, data in branches.items():
    s = max(0, data['stock'] - data['target'])
    h = max(0, data['target'] - data['stock'])
    surplus[b] = s
    shortage[b] = h

transfers = []
for s_branch, s_qty in surplus.items():
    for d_branch, d_qty in shortage.items():
        if s_qty == 0 or d_qty == 0:
            continue
        if transfer_limit == 0:
            break
        transfer_amt = min(s_qty, d_qty, transfer_limit)
        transfers.append((s_branch, d_branch, transfer_amt))
        surplus[s_branch] -= transfer_amt
        shortage[d_branch] -= transfer_amt
        transfer_limit -= transfer_amt
        s_qty = surplus[s_branch]
        d_qty = shortage[d_branch]

updated = {b: {'stock': branches[b]['stock'] + surplus[b] - shortage[b],
               'target': branches[b]['target']} for b in branches}

print("Transfer list:", transfers)
print("Updated stock levels:", updated)
print("Remaining surplus:", surplus)
print("Remaining shortage:", shortage)
```

Output (Example)

```
Transfer list:
[('A', 'B', 20)]

Updated stock levels:
{'A': {'stock': 100, 'target': 100},
 'B': {'stock': 60, 'target': 80},
 'C': {'stock': 90, 'target': 100}}

Remaining surplus:
{'A': 0, 'B': 0, 'C': 0}

Remaining shortage:
{'A': 0, 'B': 20, 'C': 10}
```

8. TEST CASE

Branches: A(120/100), B(40/80), C(90/100)
Transfer limit: 30 units per transfer.

9. CONCLUSION

The program efficiently balances inventory across branches by calculating surplus and shortage, applying transfer limits, and generating an optimal transfer plan while providing clear output and remaining quantities.

QUESTION 89: UNIVERSITY HOSTEL ROOM ALLOCATION

1. PROBLEM STATEMENT

Assign eligible students to hostel rooms while respecting room capacity and permitted student groups. Detect duplicate student assignments and report students who cannot be placed.

2. OBJECTIVES

Read student eligibility, group, and room capacity. Place each eligible student in the first compatible room with space. Identify duplicate assignments and list unallocated students. Display the final room allocation and summary.

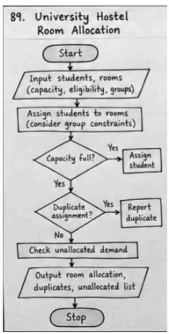
3. ASSUMPTIONS

Each student has a unique ID, name, group, and eligibility status. Each room has a capacity and accepted groups. Students are considered in order and assigned to the first compatible room with a free bed. Repeated IDs are reported and skipped.

4. ALGORITHM

1. Prepare the student list and room capacities. 2. Check each student ID for duplicates. 3. Skip duplicates and ineligible students. 4. Find the first compatible room with space. 5. Assign the student or mark them unallocated. 6. Display assignments and summary lists. 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
   FOR each student:
      IF ID is repeated: record duplicate; continue
      IF student is ineligible: record unallocated; continue
      Find first compatible room with free capacity
      IF found: assign student
      ELSE: record unallocated
   DISPLAY room assignments, duplicates, unallocated students
END
```

7. PYTHON SOURCE CODE

```
students = [
    ("S101", "Asha", "G1", True),
    ("S102", "Ravi", "G2", True),
    ("S103", "Mina", "G1", True),
    ("S104", "Omar", "G3", False),
    ("S101", "Asha", "G1", True),  # duplicate ID
]
rooms = {"Room 1": (2, {"G1"}), "Room 2": (1, {"G2"})}
assigned = {room: [] for room in rooms}
seen, duplicates, unallocated = set(), [], []

for sid, name, group, eligible in students:
    if sid in seen:
        duplicates.append(sid)
        continue
    seen.add(sid)
    if not eligible:
        unallocated.append(name)
        continue
    for room, (capacity, groups) in rooms.items():
        if group in groups and len(assigned[room]) < capacity:
            assigned[room].append(name)
            break
    else:
        unallocated.append(name)

print("Room allocation:", assigned)
print("Unallocated:", unallocated)
print("Duplicate IDs:", duplicates)
```

8. TEST CASES

Eligible G1 student with space → assigned to Room 1. Room at capacity → next compatible room is tried; if none fits, student is unallocated. Ineligible student → unallocated. Repeated ID → recorded as duplicate and skipped.

9. SAMPLE OUTPUT

```
Room allocation: {'Room 1': ['Asha', 'Mina'], 'Room 2': ['Ravi']}
Unallocated: ['Omar']
Duplicate IDs: ['S101']
```

10. CONCLUSION

The program assigns eligible students to compatible rooms within capacity, and reports duplicate IDs and students who could not be allocated.

QUESTION 90: INTEGRATED CAMPUS ENERGY AND WATER AUDIT

1. PROBLEM STATEMENT

Review energy and water consumption for campus buildings. Calculate current utility cost and percentage change, flag usage increases above a threshold, and rank buildings by an audit priority score.

2. OBJECTIVES

Compare current use with previous use for energy and water. Estimate cost using supplied rates. Identify buildings with excessive increases and list buildings from highest to lowest priority.

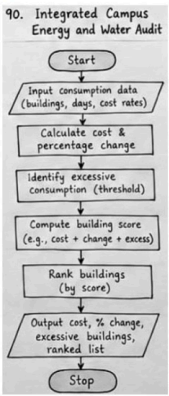
3. ASSUMPTIONS

Each building record contains previous and current energy and water use. Rates are cost per unit. A building is flagged when either utility increases by more than 15%. The illustrative priority score is current cost plus 10 points per percentage point of positive increase for each utility, plus a 1,000-point flag bonus; it is only a ranking aid.

4. ALGORITHM

1. Read building consumption and utility rates. 2. For each building, calculate previous and current cost. 3. Calculate energy and water percentage changes. 4. Flag if either increase exceeds the threshold. 5. Calculate the priority score. 6. Sort buildings by score and display the audit.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  FOR each building:
    calculate current utility cost
    calculate energy and water percentage change
    flag if either change is above threshold
    calculate priority score
  SORT buildings by score, highest first
  DISPLAY cost, changes, flags, and ranking
END
```

7. PYTHON SOURCE CODE

```
buildings = [
    ("Library", 1000, 1200, 250, 300),
    ("Hostel",   800,  760, 400, 460),
    ("Lab",     1500, 1900, 200, 260),
]  # name, old/new energy, old/new water
energy_rate, water_rate = 8, 2
limit = 15
results = []

for name, old_e, new_e, old_w, new_w in buildings:
    e_change = (new_e - old_e) / old_e * 100
    w_change = (new_w - old_w) / old_w * 100
    cost = new_e * energy_rate + new_w * water_rate
    high = e_change > limit or w_change > limit
    score = cost + 10 * (max(0, e_change) + max(0, w_change))
    if high:
        score += 1000
    results.append((name, cost, e_change, w_change, high, score))

for row in results:
    print(row)
print("Ranking:", [r[0] for r in sorted(results, key=lambda r: r[5], reverse=True)])
```

8. TEST CASES

Library: energy and water rise 20% → flagged. Hostel: energy falls 5%, water rises 15% → not flagged because the rule is strictly above 15%. Lab: both utilities rise above 15% → flagged and ranked highest.

9. SAMPLE OUTPUT

```
('Library', 10200, 20.0, 20.0, True, 11600.0)
('Hostel', 7000, -5.0, 15.0, False, 7150.0)
('Lab', 15720, 26.7, 30.0, True, 17286.7)
Ranking: ['Lab', 'Library', 'Hostel']
```

10. CONCLUSION

The audit compares utility use and costs, flags unusually large increases, and ranks buildings for follow-up. Rates and scoring weights can be changed to match campus policy.